Implement Client Script and UI Policy Incident

Coding and Solution

Prepared for the ServiceNow Incident client-side controls micro project

ServiceNow Incident micro project that uses a UI Policy and Client Scripts to enforce high-impact Incident controls, improve data quality, and prevent invalid submissions.

Project record

Template area	Completed project content
Context and need	onChange: when newValue is 1, call g_form.setValue('urgency', '1') and show an information message.
Selected approach	onSubmit: if impact is 1 and assigned_to is empty, show an Assigned To error and return false.
Implementation detail	onCellEdit: alert that State cannot be changed by list editing and call callback(false).
Expected outcome	UI Policy: apply High impact field rules and reverse the changes when Impact is no longer High.

Practical interpretation

The controls are intentionally conditional. When an agent marks an Incident as High impact, the form helps them complete the record correctly instead of relying on memory alone. When the impact is changed back to Medium or Low, the UI Policy reverses the extra restrictions so that the normal Incident workflow remains usable.

This balance is important in a service desk setting: urgent records need stronger guidance, while routine records should not be burdened with unnecessary fields or messages.

Implement Client Script and UI Policy Incident

Coding and Solution - Implementation Notes

How this template applies to the Incident configuration

Work item	Project-specific evidence	Acceptance check
Context and need	Configuration is documented in the ServiceNow Incident table and related System UI records.	Record is configured and active.
Selected approach	The rule is applied only when the form condition is true, avoiding an always-on restriction.	Condition responds to Impact = High.
Implementation detail	The expected browser behavior can be observed directly during a new Incident test.	Agent receives the expected field behavior or message.
Expected outcome	The result supports more reliable triage, routing, and SLA reporting.	A valid record can be saved after the required action.

Validation notes

Testing should begin with a new Incident. Set Impact to High, leave Assigned To empty, and submit the form. The save must be blocked and the user must see a clear error. Next, populate Assigned To and submit again to confirm the record saves. Finally, change Impact away from High and verify that the reversible UI Policy behavior has been removed.

For list editing, double-click State from the Incident list. The onCellEdit script should show its warning and reject the edit. Opening the same Incident form and updating State there should still be allowed. These tests demonstrate that the design protects data quality without preventing legitimate work.

Client Script excerpt

```
function onSubmit() {
if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') {
g_form.showErrorBox('assigned_to', 'Assigned To is mandatory for High impact incidents.');
return false;
}
return true;
}
```